

Adopting Lightspeed's DeepAgents runtime in Konveyor

November 11, 2026

Runtime integration

- Konveyor with Goose
- The prototype with Lightspeed
- Changes in Lightspeed's runtime
- Coolstore planning demo
- Questions

Konveyor with Goose

1. **You choose an app and a task**

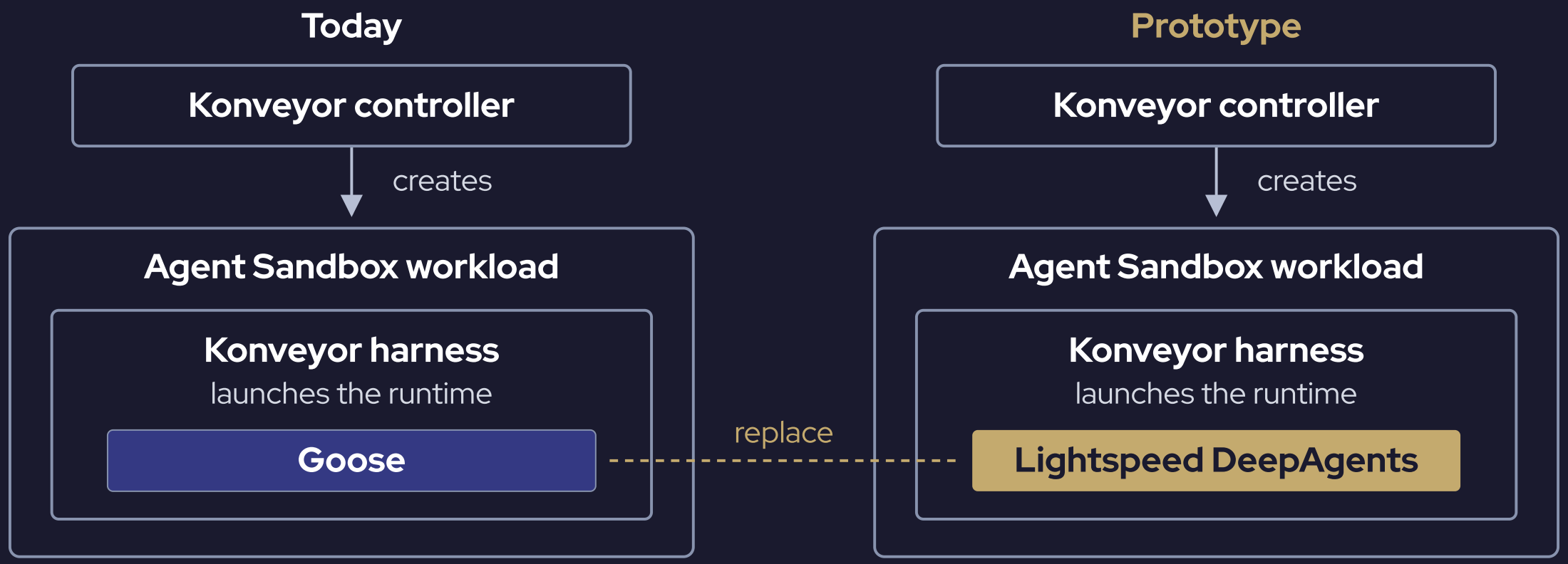
2. **Our harness prepares the repo**

3. **Goose works on the code**
   Calls the model, uses tools, commits locally

4. **Our harness verifies and pushes**
   You get the result on a Git branch



Konveyor with Lightspeed



Our harness still prepares the repo, verifies the work and pushes to Git.

Prompt completion and session lifetime

Upstream: `DeepAgentsProvider.query()` creates a graph, model clients and a new conversation thread for each query.

Our change: `RuntimeSession` keeps the graph, checkpoints and clients across prompts. Finishing a prompt leaves the session available.

Why Konveyor needs it: after a turn limit, the harness can ask the same conversation to write its handoff before verification and Git push.

Components: DeepAgents provider, RuntimeSession

Human interaction through runtime callbacks

Upstream: tools have no session-level host callback for approval or questions.

Our change: tool middleware asks for permission immediately before execution. A local `ask_user` tool calls a separate human-input callback.

Why Konveyor needs it: tool approval and agent-initiated questions remain independent controls. The session interface works with any host that implements those callbacks.

Components: session tool middleware, question callback

ACP stays outside the session core

Upstream: the batch entrypoint consumes provider events; it has no ACP session endpoint.

Our change: a separate, optional ACP adapter translates session events and callbacks into the messages Konveyor already uses.

Connection path: the UI connects to the Go harness's ACP tee, which connects to the local Lightspeed adapter. `RuntimeSession` owns the conversation.

Components: ACP adapter, session core

Steering and cancellation during a prompt

Upstream: batch cancellation exists, but there is no active-session steering interface.

Our change: guidance is queued before the next main-model call. Cancellation awaits active graph/tool work and kills and reaps local shell process groups.

Why Konveyor needs it: a user can redirect or stop work, then the harness can continue wind-down with consistent tool history.

Components: RuntimeSession controls, session shell backend

Usage reporting and turn-limit handoff

Upstream: token totals are returned at query completion; `max_turns` sets a graph recursion limit.

Our change: model-completion callbacks publish cumulative session tokens. An explicit main-model counter enforces each prompt's turn allowance.

Why Konveyor needs it: the harness gets a limit reason and can send a handoff prompt with the remaining run budget, in the same session.

Components: generation callback, session budget, Konveyor harness

Tool-output inspection in the session path

Prototype baseline: untrusted tool output is inspected before entering model context and before release to viewers.

Our change: keep both gates through sessions and ACP. Add trust for registered human answers; ignore native thinking blocks when parsing classifier decision text.

Upstream update: newer tool-output redaction and shaping work must be reconciled when this prototype is ported forward.

Components: inspection middleware, DeepAgents event release

Model settings and incomplete generations

Upstream: Anthropic wrappers accept thinking settings and output caps, but do not forward normalized reasoning effort.

Our change: forward effort into `output_config`; allow an explicit main-model output allowance; report known output cutoffs before tools execute.

Why Konveyor needs it: chosen model settings take effect, and a reported output cutoff is a failure instead of apparent completion.

Components: Anthropic model factory, output-limit middleware

Coolstore planning demo

A Java EE application we want to move to Quarkus

Play the separate demo video (2:44)

Code and result:

- Konveyor harness integration: PR #345 · source changes
- Lightspeed runtime changes: djzager/lightspeed-agentic-sandbox
- Konveyor UI changes: djzager/tackle2-ui
- Reviewed plan: djzager/coolstore, docs/plan.md

Questions

Lightspeed as the execution backend

Today: Konveyor's `AgentRun` reconciler creates a Sandbox directly. Our Go harness owns preparation, agent execution and finalization.

Candidate integration: delegate execution to a Lightspeed `AgenticRun`; its operator owns the SandboxClaim and workload lifecycle.

Adoption bar: per-run image/startup control, complete access to inputs and credentials, and preservation of harness finalization and outcomes.

Agent Sandbox remains underneath Lightspeed's SandboxClaim path. ADR 0021 separates this qualification from runtime reuse.

Roadblock: selecting the workload image

Current: one shared `sandbox-pod-spec` supplies the base image. Agent and AgenticRun have no per-invocation image selector.

- Java EE to Quarkus needs a JDK and Maven in the workload.
- A future Go Migrator needs the Go toolchain.
- Both images must be usable in concurrent runs.

Required: select the image per invocation and start its Konveyor harness.

Possible contract: a per-run image override or template selection, honoring the image's entrypoint. ADR 0021 makes image/startup control a firm requirement.

Problem: workload placement and access

Current: Lightspeed watches runs across namespaces but creates workloads in its operator namespace. Agent/LLMProvider are cluster-scoped; Secrets are namespaced.

Problem: the workload needs Hub's run-scoped inputs/token, model credentials and a reachable ACP endpoint for Konveyor's UI.

Options to qualify: run near Hub and bind provider credentials there, or run centrally and deliver Hub inputs/token with the required network access.

Components: Hub, provider/Secret binding, PodSpecBuilder and workload networking. ADR 0021 leaves placement open.

Problem: harness input and run completion

Current: Lightspeed mounts batch inputs and a Result template under `/input`; its operator uses the Pod and a completed Kubernetes Result to determine success.

Adapter needed: our harness must consume those inputs, prepare the repo, run the agent, verify/push, clean up and publish the backend Result before exiting.

Why: an agent finishing its answer is earlier than the workload finishing.

If push succeeds but Result publication fails, preserve the work and report the integration failure. ADR 0021 requires that distinction.

Problem: stopping the agent or its workload

Runtime cancellation: stops active prompt/tool work inside the Pod; the Go harness can still finalize.

Operator hard stop: terminates the workload and may prevent handoff, verification, push, token cleanup and Result publication.

Needed: an agreed normal stop path and time to finalize within the workload deadline, with emergency termination handled explicitly.

Discussion: can operator cancellation request harness wind-down before killing the workload? A per-run hard-stop proposal is open in PR #588.

OpenShell roadmap and integration contract

Roadmap question: are you planning OpenShell in this execution path, and on what timeline?

If so: which layer would own workload image/startup, placement, network policy and credential binding?

Integration impact: Hub/Git/model/MCP/ACP access and Kubernetes Result publication need a supported contract under that backend.

Should we qualify today's batch contract, or align on one that remains stable through that change?

ADRs 0016 and 0021 keep OpenShell deferred and preserve today's Gateway selection contract.

Questions